



TRƯỜNG ĐẠI HỌC QUY NHƠN

TIỂU LUẬN HỌC PHẦN
LÝ THUYẾT TỐI ƯU

THUẬT TOÁN DI TRUYỀN:
CƠ SỞ LÝ THUYẾT VÀ CÁC
ỨNG DỤNG

Trình độ đào tạo:	Thạc sĩ
Học viên thực hiện:	Nguyễn Hồ Bảo Thiên
Lớp:	Khoa học dữ liệu K28B
Giảng viên hướng dẫn:	TS. Nguyễn Văn Vũ

Gia Lai, 2026

Mục lục

1	Cơ sở lý thuyết về Thuật toán Di truyền	4
1.1	Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa	4
1.2	Cơ sở Khoa học và Nguyên lý Hoạt động của GA	6
1.3	Các Thành phần và Phép toán Cốt lõi	7
1.4	Quy trình Thuật toán và Điều kiện Dừng	11
1.5	Lập trình Di truyền	13
2	Ứng dụng: Thực nghiệm trên Bốn Bài toán	15
2.1	Tối ưu không ràng buộc	15
2.2	Tối ưu có ràng buộc: Bộ chuẩn CEC 2006	23
2.2.1	Bài toán g01	24
2.2.2	Bài toán g06	25
2.2.3	Bài toán g08	26
2.2.4	Bài toán g11	26
2.2.5	Bài toán g15	27
2.2.6	Nhận xét chung	28
2.3	Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ	29
2.4	Hồi quy Ký hiệu bằng Lập trình Di truyền	30
2.5	Tổng kết Chương	31
	Tài liệu tham khảo	33

Danh sách hình vẽ

1.1	Chu trình tiến hóa của Thuật toán Di truyền	12
2.1	Hàm Easom: đồ thị trong không gian ba chiều và diễn biến giá trị tốt nhất qua các thế hệ. Đường nét đứt đánh dấu thế hệ mà GA chốt được nghiệm tốt nhất.	16
2.2	Hàm Rastrigin: mạng lưới cực tiểu cục bộ dày đặc quanh cực tiểu toàn cục, và diễn biến giá trị tốt nhất qua các thế hệ.	17
2.3	Hàm Rosenbrock: thung lũng cong hẹp hình quả chuối, và diễn biến giá trị tốt nhất qua các thế hệ — GA chốt kỷ lục ngay ở thế hệ 11 rồi không cải thiện thêm trong 89 thế hệ còn lại.	19
2.4	Hàm Ackley: phễu lớn ở thang toàn cục phủ gợn sóng cực tiểu cục bộ ở thang nhỏ, và diễn biến giá trị tốt nhất qua các thế hệ.	20
2.5	Hàm Schwefel: cực tiểu toàn cục nằm gần biên miền tìm kiếm, xa vùng trông có vẻ tốt quanh gốc tọa độ, và diễn biến giá trị tốt nhất qua các thế hệ.	21

Danh sách bảng

1.1	Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền	6
2.1	Kết quả GA trên năm hàm benchmark không ràng buộc	22
2.2	Đặc điểm năm bài toán được chọn từ bộ chuẩn CEC 2006	23
2.3	Kết quả trên bài toán g01	25
2.4	Kết quả trên bài toán g06	25
2.5	Kết quả trên bài toán g08	26
2.6	Kết quả trên bài toán g11	27
2.7	Kết quả trên bài toán g15	28
2.8	Tổng hợp kết quả GA trên năm bài toán chuẩn CEC 2006	28
2.9	Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95	30
2.10	Kết quả Hồi quy Tuyến tính và GP trên hai đại lượng vật lý phái sinh	31

Chương 1

Cơ sở lý thuyết về Thuật toán Di truyền

1.1 Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa

Bài toán Tối ưu hóa và Không gian Tìm kiếm

Một bài toán tối ưu hóa tổng quát yêu cầu tìm một vector biến quyết định $x = (x_1, x_2, \dots, x_n)$ sao cho một hàm mục tiêu $f(x)$ đạt giá trị cực tiểu hoặc cực đại, trong khi x vẫn thỏa mãn một tập ràng buộc xác định miền khả thi Ω :

$$\min_{x \in \Omega} f(x) \quad \text{hoặc} \quad \max_{x \in \Omega} f(x). \quad (1.1)$$

Miền $\Omega \subseteq \mathbb{R}^n$, còn gọi là không gian tìm kiếm, thường được xác định bởi một hệ ràng buộc đẳng thức và bất đẳng thức trên các biến quyết định. Bài toán được coi là **có ràng buộc** khi Ω là một tập con thực sự của không gian biến, và **không ràng buộc** khi Ω trùng với toàn bộ không gian biến.

Hai thách thức lớn khiến tối ưu hóa trở thành một bài toán khó nói chung là:

- **Cực trị cục bộ và cực trị toàn cục.** Khi hàm mục tiêu không lồi hoặc đa cực trị, một thuật toán chỉ dựa vào thông tin cục bộ quanh điểm hiện tại — ví dụ đạo hàm — có xu hướng hội tụ về cực trị cục bộ gần điểm khởi tạo nhất, thay vì cực trị toàn cục của toàn miền Ω . Đây chính là hạn chế cốt lõi của các phương pháp tối ưu dựa trên gradient khi áp dụng cho bài toán không lồi.
- **Bài toán NP-hard và sự bùng nổ tổ hợp.** Với nhiều lớp bài toán tối ưu tổ hợp như bài toán người bán hàng TSP hay bài toán lập lịch, số lượng phương án khả thi tăng theo hàm mũ hoặc giai thừa của kích thước bài toán, khiến việc

duyet toàn bộ không gian tìm kiếm trở nên bất khả thi về mặt tính toán ngay cả với các bài toán kích thước vừa phải.

Hai thách thức trên là động lực chính cho sự phát triển của các phương pháp tìm kiếm không dựa trên đạo hàm, có khả năng khám phá đồng thời nhiều vùng của không gian tìm kiếm — trong đó có họ thuật toán tiến hóa được trình bày ở mục tiếp theo.

Họ Thuật toán Tiến hóa (Evolutionary Computation)

Khác với các phương pháp tối ưu cổ điển dựa trên đạo hàm như Gradient Descent hay phương pháp Newton, vốn chỉ xử lý một điểm duy nhất tại mỗi bước lặp, **metaheuristic** là các chiến lược tìm kiếm ở mức tổng quát, không đảm bảo tìm được nghiệm tối ưu toàn cục tuyệt đối nhưng thường tìm được nghiệm đủ tốt trong thời gian chấp nhận được. Metaheuristic đặc biệt hữu ích khi hàm mục tiêu không khả vi, có nhiễu, không lồi, hoặc thậm chí không có biểu thức giải tích tường minh.

Một nhánh quan trọng của metaheuristic là các **thuật toán tìm kiếm dựa trên quần thể**: thay vì duy trì một điểm duy nhất, thuật toán duy trì cả một tập hợp điểm và để chúng cùng tiến hóa qua các thế hệ, nhờ đó khảo sát đồng thời nhiều vùng của không gian tìm kiếm và giảm nguy cơ mắc kẹt tại một cực trị cục bộ duy nhất. **Họ Thuật toán Tiến hóa** (Evolutionary Algorithms — EA) là đại diện tiêu biểu nhất của nhóm này, mô phỏng nguyên lý chọn lọc tự nhiên của Darwin để dẫn dắt quá trình tìm kiếm. Họ EA bao gồm bốn nhánh chính, phân biệt nhau chủ yếu ở cách biểu diễn giải pháp và toán tử tiến hóa được nhấn mạnh:

- **Thuật toán Di truyền (Genetic Algorithm — GA)**: nhấn mạnh vai trò của lai ghép như toán tử tìm kiếm chính, kết hợp thông tin từ nhiều cá thể cha mẹ để tạo ra cá thể con; đây là đối tượng nghiên cứu chính của chương này.
- **Chiến lược Tiến hóa (Evolution Strategies — ES)**: do Rechenberg và Schwefel phát triển, tập trung vào tối ưu tham số thực với đột biến Gauss là toán tử chính và có cơ chế tự thích nghi các tham số đột biến trong quá trình tiến hóa.
- **Lập trình Tiến hóa (Evolutionary Programming — EP)**: do Fogel đề xuất, tiến hóa chủ yếu thông qua đột biến trên biểu diễn hành vi của cá thể, gần như không sử dụng lai ghép.
- **Lập trình Di truyền (Genetic Programming — GP)**: do Koza phát triển, mở rộng GA để tiến hóa trực tiếp các cấu trúc cây biểu thức hoặc chương trình máy tính, thường dùng để tự động khám phá công thức toán học hoặc chương trình giải quyết một tác vụ cho trước.

Trong phạm vi tiểu luận này, trọng tâm là Thuật toán Di truyền áp dụng cho bài toán tối ưu số thực có ràng buộc, với cách mã hóa và các toán tử cụ thể được trình bày trong các mục tiếp theo.

1.2 Cơ sở Khoa học và Nguyên lý Hoạt động của GA

Nguồn gốc Lịch sử và Nguyên lý Sinh học

Thuật toán Di truyền được John Holland giới thiệu năm 1975 tại Đại học Michigan trong công trình *Adaptation in Natural and Artificial Systems* [1], và sau đó được David Goldberg hệ thống hoá, phổ biến rộng rãi hơn qua cuốn sách kinh điển năm 1989 *Genetic Algorithms in Search, Optimization, and Machine Learning* [2]. Ý tưởng nền tảng của GA là mô phỏng ba nguyên lý cốt lõi trong học thuyết tiến hóa của Darwin:

- **Chọn lọc tự nhiên** (“*Survival of the Fittest*”): cá thể thích nghi tốt hơn với môi trường có xác suất sống sót và sinh sản cao hơn.
- **Di truyền**: đặc điểm của cá thể cha mẹ được truyền lại cho thế hệ con thông qua vật chất di truyền.
- **Biến dị**: các sai khác ngẫu nhiên xuất hiện giữa các cá thể — do lai ghép và đột biến — là nguồn tạo ra tính đa dạng cần thiết để quá trình chọn lọc có tác dụng.

GA chuyển ba nguyên lý sinh học trên thành một quy trình tính toán bằng cách xây dựng một phép tương ứng giữa các khái niệm sinh học và các thành phần thuật toán, được tóm tắt trong Bảng 1.1.

Bảng 1.1: Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền

Sinh học	Thuật ngữ tiếng Anh	Thuật toán Di truyền
Nhiễm sắc thể	Chromosome	Mã hóa giải pháp
Gen	Gene	Một biến thành phần trong giải pháp
Cá thể	Individual	Một giải pháp ứng viên
Quần thể	Population	Tập hợp các giải pháp tại một thế hệ
Độ thích nghi	Fitness	Giá trị hàm độ thích nghi
Thế hệ	Generation	Một vòng lặp tiến hóa

Nhờ phép tương ứng này, một bài toán tối ưu bất kỳ có thể được diễn dịch lại thành một quá trình “tiến hóa nhân tạo”: xuất phát từ một quần thể giải pháp ngẫu nhiên, để các giải pháp tốt hơn có nhiều cơ hội “sinh sản” hơn qua nhiều thế hệ, quần thể dần dịch chuyển về phía các vùng có giá trị hàm mục tiêu tốt của không gian tìm kiếm.

Phương pháp Mã hóa Giải pháp

Bước đầu tiên khi áp dụng GA cho một bài toán cụ thể là chọn cách mã hóa một giải pháp dưới dạng “nhiễm sắc thể” mà các toán tử di truyền có thể thao tác được. Bốn cách mã hóa phổ biến nhất là:

Mã hóa Nhị phân (Binary Encoding): giải pháp được biểu diễn bằng một chuỗi bit 0 và 1, ví dụ 10110010. Đây là cách mã hóa nguyên thủy nhất trong GA cổ điển của Holland, phù hợp với các bài toán rời rạc hoặc logic, nhưng khi áp dụng cho biến số thực cần một bước rời rạc hóa làm giảm độ chính xác.

Mã hóa Số thực (Real-value Encoding): giải pháp là một dãy số thực, ví dụ [3.14, -0.05, 12.8], mỗi phần tử tương ứng trực tiếp với một biến quyết định. Cách mã hóa này phù hợp tự nhiên với các bài toán tối ưu hóa liên tục nhiều biến, tránh được sai số lượng tử hóa của mã hóa nhị phân, và là cách mã hóa được sử dụng trong phần cài đặt thực nghiệm của tiểu luận ở Chương 2.

Mã hóa Hoán vị (Permutation Encoding): giải pháp là một thứ tự sắp xếp của một tập hợp phần tử, ví dụ [3, 1, 4, 2, 5]. Đây là cách mã hóa tự nhiên cho các bài toán định tuyến/lập lịch như bài toán người bán hàng (TSP) hay bài toán phân công công việc, nơi lời giải về bản chất là một hoán vị.

Mã hóa Dựa trên Cấu trúc (Tree/Graph Encoding): giải pháp được biểu diễn dưới dạng cây biểu thức hoặc đồ thị, sử dụng chủ yếu trong Lập trình Di truyền để tiến hóa các công thức hoặc chương trình.

Việc lựa chọn cách mã hóa quyết định trực tiếp các toán tử lai ghép và đột biến nào có thể áp dụng được, như sẽ trình bày ở Mục 1.3.

1.3 Các Thành phần và Phép toán Cốt lõi

Hàm độ thích nghi và Ràng buộc

Hàm độ thích nghi đóng vai trò đánh giá chất lượng của từng cá thể trong quần thể, làm cơ sở cho phép chọn lọc. Với bài toán tối ưu không ràng buộc, hàm thích nghi thường được xây dựng trực tiếp từ hàm mục tiêu, ví dụ đảo dấu hàm mục tiêu khi bài toán gốc là bài toán cực tiểu hóa nhưng GA nguyên thủy được phát biểu dưới dạng cực đại hóa độ thích nghi.

Đối với bài toán **có ràng buộc** — trọng tâm của tiểu luận này — cách tiếp cận kinh điển là chuyển ràng buộc thành một số hạng phạt cộng vào hàm mục tiêu, tạo

thành hàm thích nghi đã phạt:

$$F(x) = f(x) \pm \sum_i \lambda_i \cdot g_i(x), \quad (1.2)$$

trong đó $g_i(x)$ đo mức độ vi phạm ràng buộc thứ i (bằng 0 nếu ràng buộc được thỏa mãn), còn $\lambda_i > 0$ là hệ số phạt tương ứng; dấu $+$ hay $-$ và độ lớn của λ_i được chọn tùy theo bài toán là cực tiểu hay cực đại hóa, sao cho cá thể vi phạm ràng buộc luôn bị đánh giá kém hơn cá thể khả thi có cùng giá trị hàm mục tiêu.

Hạn chế cố hữu của hàm phạt tĩnh là hệ số λ_i phải được chọn thủ công và cố định trong suốt quá trình tiến hóa: chọn λ_i quá nhỏ khiến quần thể “phớt lờ” ràng buộc, trong khi chọn quá lớn khiến số hạng phạt lấn át hoàn toàn hàm mục tiêu, làm quần thể chỉ tập trung thỏa mãn ràng buộc mà mất khả năng phân biệt các nghiệm khả thi tốt và xấu. Nhiều phương pháp xử lý ràng buộc nâng cao hơn đã được đề xuất để khắc phục hạn chế này — ví dụ quy tắc khả thi của Deb hay các ngưỡng ε giảm dần theo thời gian do Takahama và Sato đề xuất — và sẽ được trình bày cụ thể cùng phần cài đặt thực nghiệm ở Chương 2.

Các Phép chọn lọc

Phép chọn lọc quyết định cá thể nào trong quần thể hiện tại được chọn làm “bố mẹ” để sinh ra thế hệ tiếp theo, thiên vị có chủ đích về phía các cá thể có độ thích nghi cao hơn.

Vòng quay Roulette (Roulette Wheel Selection): xác suất chọn cá thể i tỉ lệ thuận với độ thích nghi f_i của nó so với tổng độ thích nghi toàn quần thể:

$$P_i = \frac{f_i}{\sum_{j=1}^N f_j}. \quad (1.3)$$

Hạn chế của phương pháp này là khi một cá thể có độ thích nghi vượt trội hẳn phần còn lại, nó sẽ chiếm phần lớn xác suất được chọn, khiến quần thể nhanh chóng mất đa dạng.

Thách đấu (Tournament Selection): chọn ngẫu nhiên k cá thể từ quần thể, sau đó cá thể tốt nhất trong nhóm k cá thể này được chọn làm bố/mẹ. Tham số k , tức kích thước thách đấu, điều khiển trực tiếp áp lực chọn lọc: k càng lớn, áp lực chọn lọc càng cao và quần thể hội tụ càng nhanh, nhưng cũng dễ mất đa dạng hơn.

Xếp hạng (Rank-based Selection): cá thể được chọn dựa trên thứ hạng của nó trong quần thể đã sắp xếp theo độ thích nghi, thay vì giá trị thích nghi tuyệt

đối, giúp tránh hiện tượng một cá thể siêu việt áp đảo toàn bộ xác suất chọn như ở Roulette Wheel.

Bên cạnh các toán tử chọn lọc bố mẹ nêu trên, một chiến lược lựa chọn cá thể sang thế hệ sau đặc biệt quan trọng là **giữ lại cá thể ưu tú**, được trình bày riêng ở phần dưới đây do vai trò và cách áp dụng của nó khác với chọn lọc bố mẹ thông thường.

Các Phép lai ghép

Lai ghép kết hợp vật chất di truyền của hai cá thể bố mẹ để tạo ra cá thể con, với tần suất lai ghép p_c thường nằm trong khoảng 0.6 đến 0.95 — nghĩa là phần lớn, nhưng không phải toàn bộ, các cặp bố mẹ được chọn sẽ thực sự lai ghép; phần còn lại được sao chép nguyên vẹn sang thế hệ con.

Đối với mã hóa nhị phân và số thực, ba toán tử phổ biến nhất là:

- **Lai ghép đơn điểm (Single-point Crossover)**: chọn ngẫu nhiên một vị trí cắt trên chuỗi nhiễm sắc thể, hai cá thể con được tạo bằng cách hoán đổi các đoạn nằm sau vị trí cắt giữa hai bố mẹ.
- **Lai ghép đa điểm (Multi-point Crossover)**: tổng quát hóa lai ghép đơn điểm với nhiều vị trí cắt, các đoạn xen kẽ giữa hai bố mẹ được hoán đổi.
- **Lai ghép đồng nhất (Uniform Crossover)**: mỗi vị trí gen được quyết định độc lập, ngẫu nhiên xem sẽ lấy từ bố hay mẹ, không phụ thuộc vào vị trí trong chuỗi.

Với mã hóa số thực, một họ toán tử quan trọng khác là lai ghép trộn (blend crossover, BLX- α) — cách tiếp cận được sử dụng trong phần cài đặt của tiểu luận. Với hai cá thể bố mẹ $p_1, p_2 \in \mathbb{R}^n$ và một hệ số trộn α được lấy mẫu ngẫu nhiên và có thể nhận cả giá trị ngoài khoảng $[0, 1]$, hai cá thể con được tạo theo tổ hợp tuyến tính:

$$c_1 = \alpha \cdot p_1 + (1 - \alpha) \cdot p_2, \quad c_2 = \alpha \cdot p_2 + (1 - \alpha) \cdot p_1. \quad (1.4)$$

Khi α được lấy mẫu từ một khoảng mở rộng ra ngoài $[0, 1]$ (ví dụ $\alpha \sim \mathcal{U}(-0.25, 1.25)$) thay vì giới hạn trong đoạn nối hai bố mẹ, cá thể con có thể nằm ngoài đoạn thẳng nối p_1 và p_2 , giúp toán tử này vừa có khả năng khai thác vùng lân cận bố mẹ, vừa giữ được khả năng khám phá ra ngoài đoạn đó thay vì chỉ co dần khoảng tìm kiếm qua các thế hệ.

Đối với mã hóa hoán vị trong các bài toán định tuyến và lập lịch, việc hoán đổi trực tiếp từng đoạn như trên có thể tạo ra cá thể con vi phạm tính hoán vị vì có phần tử xuất hiện nhiều lần hoặc bị thiếu, nên cần các toán tử chuyên biệt:

- **PMX (Partially Matched Crossover):** hoán đổi một đoạn con giữa hai bố mẹ như lai ghép đơn điểm, sau đó sửa các vị trí còn lại theo một ánh xạ tương ứng từng phần để đảm bảo tính hoán vị.
- **OX (Order Crossover):** giữ nguyên một đoạn con từ bố mẹ thứ nhất, các phần tử còn lại được điền theo đúng thứ tự xuất hiện của chúng ở bố mẹ thứ hai.
- **CX (Cycle Crossover):** xác định các “chu trình” giữa vị trí phần tử ở hai bố mẹ, mỗi cá thể con được tạo bằng cách giữ nguyên các chu trình xen kẽ từ mỗi bố mẹ.

Các Phép đột biến

Đột biến tạo ra biến dị ngẫu nhiên trên một cá thể, với tần suất đột biến p_m trên mỗi gen thường nhỏ, nằm trong khoảng 0.001 đến 0.05 đối với GA nhị phân cổ điển; vai trò chính của đột biến là duy trì đa dạng di truyền và giúp thuật toán thoát khỏi các vùng mà lai ghép một mình không thể tiếp cận được.

- **Đột biến đảo bit (Bit-flip Mutation):** với mã hóa nhị phân, mỗi bit được đảo giá trị ($0 \leftrightarrow 1$) độc lập với xác suất p_m .
- **Đột biến Gauss / Uniform:** với mã hóa số thực, mỗi biến x_i được cộng thêm một nhiễu ngẫu nhiên, phổ biến nhất là nhiễu Gauss:

$$x'_i = x_i + \mathcal{N}(0, \sigma_i^2), \quad (1.5)$$

trong đó độ lệch chuẩn σ_i thường được đặt tỉ lệ với độ rộng miền giá trị của biến x_i , sao cho bước đột biến có ý nghĩa tương đối giống nhau giữa các biến có thang đo khác nhau.

- **Đột biến hoán đổi, đảo ngược, chèn (Swap, Inversion, Insertion):** dành cho mã hóa hoán vị — lần lượt là hoán đổi vị trí hai phần tử, đảo ngược thứ tự một đoạn con, và di chuyển một phần tử sang vị trí khác trong chuỗi — mỗi thao tác đều bảo toàn tính hoán vị của cá thể.

Chiến lược giữ lại Cá thể Ưu tú (Elitism)

Vì chọn lọc, lai ghép và đột biến đều mang tính ngẫu nhiên, không có gì đảm bảo cá thể tốt nhất của thế hệ hiện tại sẽ xuất hiện lại ở thế hệ kế tiếp — trên lý thuyết, độ thích nghi tốt nhất toàn quần thể hoàn toàn có thể giảm đi giữa hai thế hệ liên tiếp. **Elitism** khắc phục vấn đề này bằng cách bắt buộc giữ lại nguyên vẹn k cá thể tốt nhất của thế hệ hiện tại, chuyển thẳng sang thế hệ sau mà không qua lai ghép hay

đột biến. Nhờ đó, độ thích nghi tốt nhất từng tìm được không bao giờ giảm qua các thế hệ. Việc chọn kích thước elitism k là một sự đánh đổi: k quá nhỏ có nguy cơ đánh mất cá thể tốt vào tay các toán tử ngẫu nhiên, còn k quá lớn làm giảm áp lực khám phá của quần thể, có thể góp phần làm giảm đa dạng của quần thể và khiến quá trình tìm kiếm hội tụ sớm về một vùng hẹp của không gian tìm kiếm.

1.4 Quy trình Thuật toán và Điều kiện Dừng

Sơ đồ khối và Giải mã

Kết hợp các thành phần đã trình bày ở Mục 1.3, một thể hệ GA điển hình lặp lại chu trình: chọn lọc bố mẹ, lai ghép, đột biến, đánh giá quần thể con, rồi cập nhật quần thể mới có áp dụng elitism — như minh họa ở Hình 1.1.

Thuật toán 1 trình bày lại chu trình này dưới dạng giả mã.

Thuật toán 1 Giải mã tổng quát của Thuật toán Di truyền

```

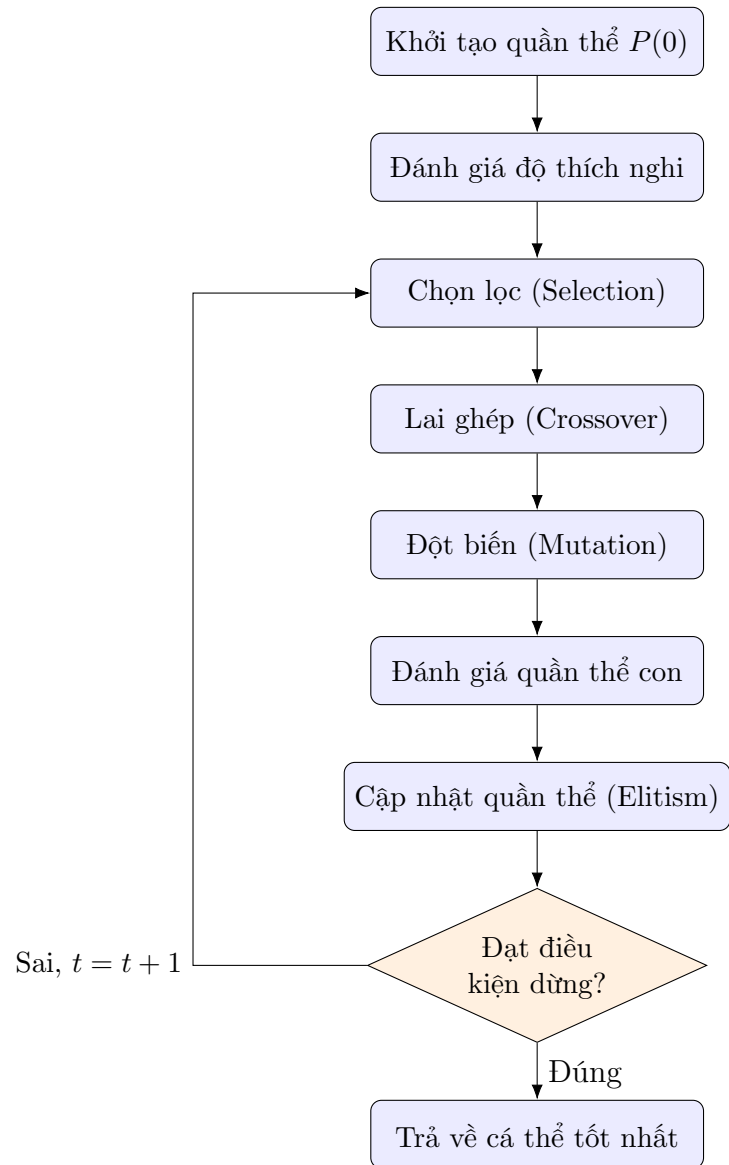
1:  $t \leftarrow 0$ 
2: Khởi tạo quần thể ban đầu  $P(0)$  ngẫu nhiên
3: Đánh giá độ thích nghi của  $P(0)$ 
4: while điều kiện dừng chưa thỏa mãn do
5:    $t \leftarrow t + 1$ 
6:   Chọn lọc tập bố mẹ  $S(t)$  từ  $P(t - 1)$ 
7:   Thực hiện lai ghép (crossover) trên  $S(t)$ , tạo tập con  $C(t)$ 
8:   Thực hiện đột biến (mutation) trên  $C(t)$ 
9:   Đánh giá độ thích nghi của  $C(t)$ 
10:   $P(t) \leftarrow$  cập nhật quần thể mới (áp dụng elitism)
11: end while
12: return cá thể tốt nhất tìm được

```

Tiêu chuẩn Dừng và Đánh giá sự Hội tụ

Vòng lặp tiến hóa cần một điều kiện dừng tường minh; ba tiêu chuẩn phổ biến, có thể kết hợp với nhau, là:

- **Đạt số lượng thế hệ tối đa G_{max} :** đơn giản và dễ kiểm soát ngân sách tính toán nhất, nhưng G_{max} quá nhỏ có nguy cơ dừng trước khi quần thể hội tụ, trong khi G_{max} quá lớn gây lãng phí tính toán không cần thiết sau khi quần thể đã hội tụ.
- **Đạt giá trị fitness mục tiêu:** dừng ngay khi tìm được một cá thể có độ thích nghi đạt hoặc vượt một ngưỡng đặt trước — phù hợp khi đã biết hoặc ước lượng được giá trị tối ưu cần đạt.



Hình 1.1: Chu trình tiến hóa của Thuật toán Di truyền

- **Quần thể hội tụ (stagnation):** dừng khi độ thích nghi của cá thể tốt nhất không cải thiện sau K thế hệ liên tiếp — dấu hiệu cho thấy các toán tử di truyền không còn tạo ra tiến triển đáng kể. Cần lưu ý tiêu chuẩn này có thể dừng thuật toán quá sớm nếu K quá nhỏ, do quần thể có thể dừng cải thiện tạm thời trước khi tiếp tục tiến triển ở các thế hệ sau.

1.5 Lập trình Di truyền

Lập trình Di truyền, do John Koza phát triển từ cuối thập niên 1980 và hệ thống hoá trong công trình kinh điển năm 1992 [8], là một mở rộng tự nhiên của GA đã giới thiệu ở Mục 1.2 dưới tên gọi *mã hóa dựa trên cấu trúc*. Khác biệt cốt lõi so với GA mã hóa số thực hay hoán vị là: thay vì tiến hóa một chuỗi giá trị có độ dài cố định trong một khuôn dạng lời giải cho trước, GP tiến hóa trực tiếp **cấu trúc** của một biểu thức hoặc chương trình, cho phép thuật toán tự khám phá đồng thời cả hình dạng lẫn tham số của lời giải.

Mã hóa dạng cây biểu thức

Mỗi cá thể trong GP là một cây biểu thức: nút trong là một toán tử lấy từ một *tập hàm* cho trước — ví dụ các phép toán số học $\{+, -, \times, \div\}$ hay hàm lượng giác $\{\sin, \cos\}$ — còn nút lá là một biến đầu vào hoặc một hằng số, lấy từ một *tập đầu cuối*. Duyệt cây theo đúng cấu trúc phân cấp của nó cho ra một biểu thức toán học tường minh, tính được giá trị trên bất kỳ bộ dữ liệu đầu vào nào — nhờ đó, một quần thể các cây khác nhau về cả hình dạng lẫn nội dung có thể được đánh giá và so sánh với nhau như trong GA thông thường.

Hàm độ thích nghi

Bài toán tiêu biểu nhất mà GP được áp dụng là **hồi quy ký hiệu**: tự động tìm một biểu thức toán học mô tả tốt một tập dữ liệu quan sát (X, y) mà không giả định trước dạng hàm cụ thể — khác hẳn hồi quy tuyến tính hay hồi quy đa thức, nơi dạng hàm đã được ấn định sẵn và chỉ các hệ số được tối ưu. Độ thích nghi của một cây trong bài toán này thường được xây dựng từ sai số dự đoán, ví dụ sai số bình phương trung bình (MSE) giữa giá trị cây tính ra và giá trị y thực tế, cộng thêm một số hạng phạt tỉ lệ với kích thước cây (*parsimony pressure*) nhằm hạn chế hiện tượng cây “phình to” không kiểm soát (bloat) mà không mang lại cải thiện tương xứng về độ chính xác.

Toán tử tiến hóa

Lai ghép và đột biến trong GP thao tác trực tiếp trên cấu trúc cây thay vì trên một chuỗi có độ dài cố định:

- **Lai ghép trao đổi cây con (Subtree Crossover):** chọn ngẫu nhiên một nút trên mỗi cây bố mẹ, rồi hoán đổi hai cây con bắt rễ tại các nút đó với nhau để tạo ra hai cây con.
- **Đột biến thay cây con (Subtree Mutation):** chọn ngẫu nhiên một nút trên cây, rồi thay toàn bộ cây con bắt rễ tại nút đó bằng một cây con mới được sinh ngẫu nhiên.

Các thành phần còn lại của chu trình tiến hóa — khởi tạo quần thể, chọn lọc, elitism, điều kiện dừng — về bản chất giống hệt GA đã trình bày ở các mục trước, chỉ khác đối tượng thao tác là một **cây** thay vì một **vector** số hay hoán vị.

Trên cơ sở các thành phần và quy trình đã trình bày ở chương này, Chương 2 sẽ trình bày cụ thể việc áp dụng GA và GP cho bốn nhóm bài toán — tối ưu không ràng buộc, tối ưu có ràng buộc, bài toán người du lịch, và hồi quy ký hiệu — đồng thời đối chiếu kết quả thực nghiệm với mốc so sánh tương ứng của từng nhóm: giá trị cực tiểu toàn cục đã biết của các hàm benchmark, nghiệm tối ưu công bố của bộ chuẩn CEC 2006 và của TSPLIB95, cùng hồi quy tuyến tính. Độc giả quan tâm đến các hướng phát triển và ứng dụng khác của GA có thể tham khảo thêm tổng quan của Katoch và cộng sự năm 2020 [5], cũng như một ứng dụng kinh điển kết hợp GA với huấn luyện mạng nơ-ron của Whitley và cộng sự năm 1990 [6].

Chương 2

Ứng dụng: Thực nghiệm trên Bốn Bài toán

Chương này trình bày việc áp dụng các nguyên lý đã trình bày ở Chương 1 vào bốn nhóm bài toán cụ thể. Mỗi nhóm sử dụng một cách mã hóa khác nhau: tối ưu không ràng buộc dùng mã hóa số thực (Mục 2.1), tối ưu có ràng buộc cũng dùng mã hóa số thực nhưng kết hợp thêm cơ chế xử lý ràng buộc (Mục 2.2), bài toán người du lịch dùng mã hóa hoán vị (Mục 2.3), và hồi quy ký hiệu dùng mã hóa dạng cây biểu thức theo nguyên lý Lập trình Di truyền (Mục 2.4). Với mỗi bài toán, phát biểu toán học đầy đủ được trình bày trước, sau đó là phương pháp so sánh được sử dụng và bảng kết quả thực nghiệm.

2.1 Tối ưu không ràng buộc

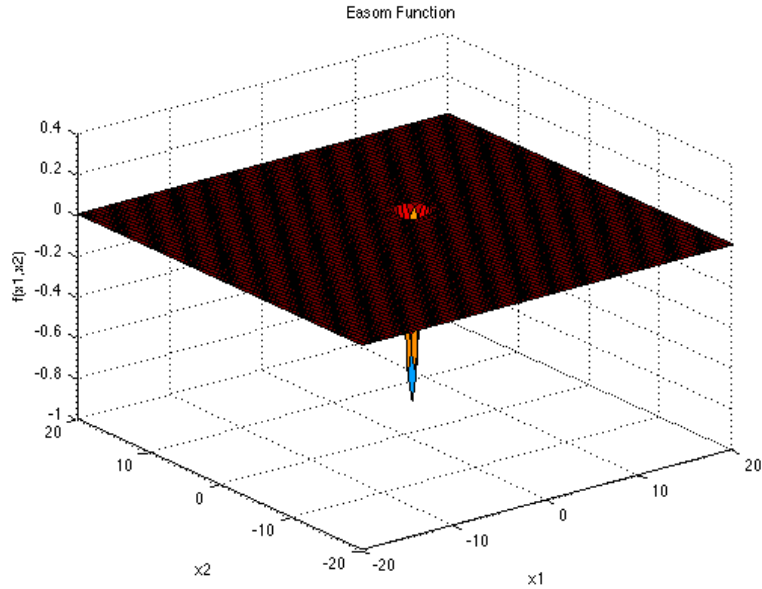
Năm hàm benchmark kinh điển của tối ưu không ràng buộc được sử dụng để đánh giá khả năng tìm cực trị toàn cục của GA. Cả năm hàm đều có hai biến x, y và một cực tiểu toàn cục đã biết trước, cho phép so sánh trực tiếp giá trị GA tìm được với giá trị đúng. Việc chọn đúng năm hàm này không ngẫu nhiên: mỗi hàm đại diện cho một kiểu địa hình gây khó khăn khác nhau, nhờ đó bộ năm hàm bao quát được các tình huống mà một thuật toán tối ưu toàn cục phải đối mặt. Với mỗi hàm, đồ thị hàm trong không gian ba chiều và đường hội tụ thực nghiệm của GA được đặt lần lượt trong cùng một hình, để đối chiếu giữa đặc điểm địa hình và hành vi hội tụ tương ứng.

Hàm Easom có công thức

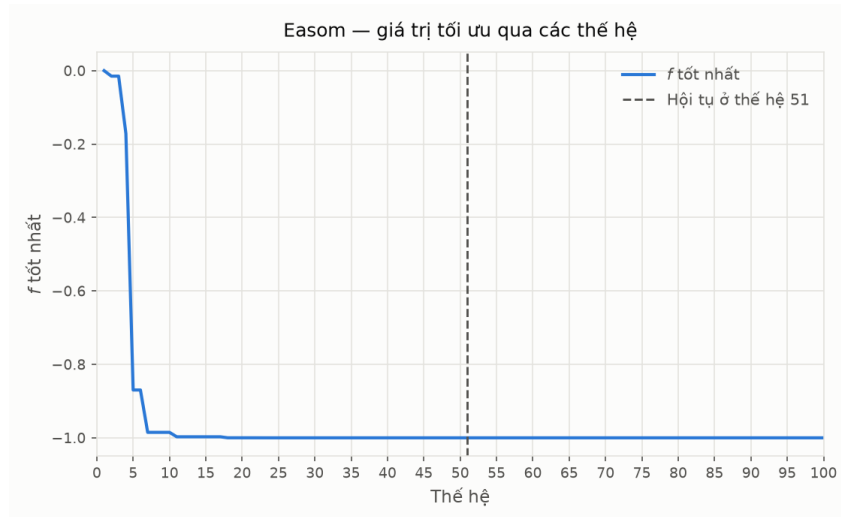
$$f(x, y) = -\cos(x) \cos(y) \exp(-(x - \pi)^2 - (y - \pi)^2), \quad x, y \in [-100, 100], \quad (2.1)$$

với cực tiểu toàn cục $f(\pi, \pi) = -1$. Khó khăn của hàm này nằm ở chỗ thừa số $\exp(-(x - \pi)^2 - (y - \pi)^2)$ tắt cực nhanh khi rời khỏi điểm (π, π) : chỉ cần cách tâm khoảng ba đơn vị, giá trị hàm đã nhỏ hơn 10^{-4} và coi như bằng 0. Vùng thực sự mang thông tin dẫn

đường vì thể chiếm chưa tới 0,1% diện tích miền tìm kiếm $[-100, 100]^2$, toàn bộ phần còn lại là một mặt phẳng gần như tuyệt đối. Đây là tình huống bất lợi nhất cho mọi phương pháp dựa trên gradient: đứng ở vùng phẳng thì đạo hàm xấp xỉ 0 theo mọi hướng, nên không có thông tin cục bộ nào chỉ ra nên đi về đâu. Hàm này vì vậy kiểm tra thuần túy khả năng *tìm ra* đúng vùng chứa nghiệm bằng cách rải điểm, chứ không phải khả năng tinh chỉnh.



(a) Hàm Easom trong không gian ba chiều



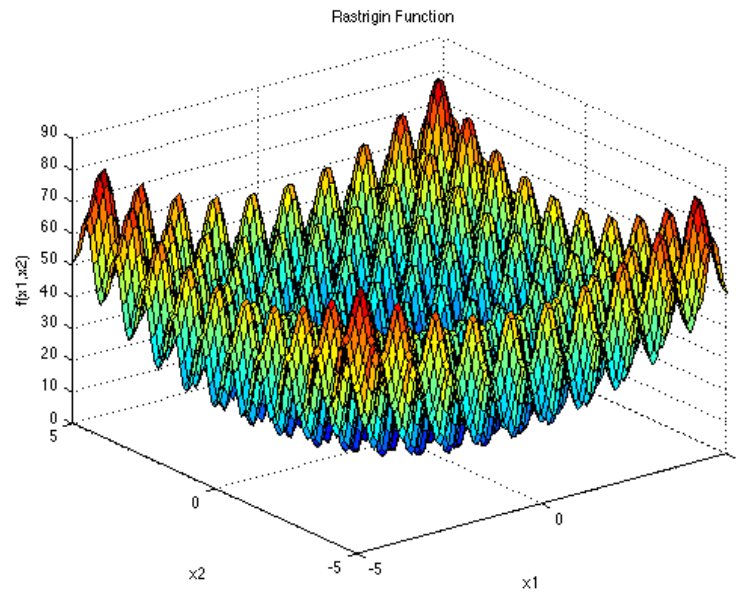
(b) Đường hội tụ của GA

Hình 2.1: Hàm Easom: đồ thị trong không gian ba chiều và diễn biến giá trị tốt nhất qua các thế hệ. Đường nét đứt đánh dấu thế hệ mà GA chót được nghiệm tốt nhất.

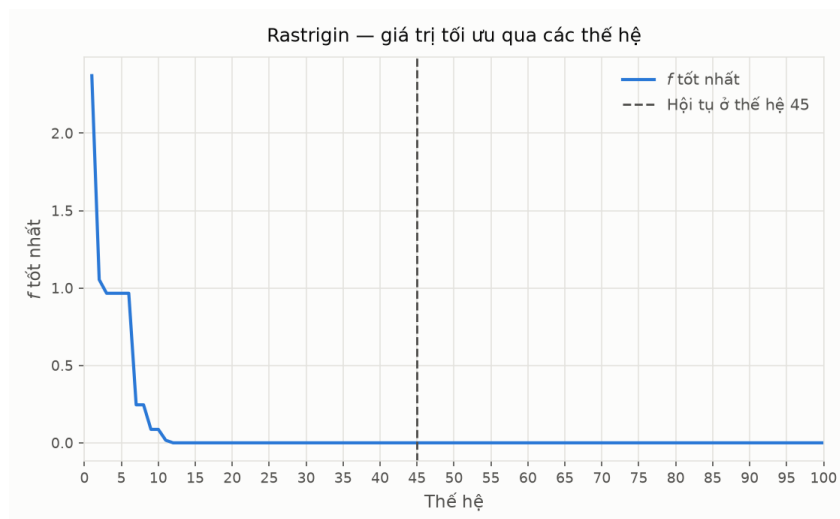
Hàm Rastrigin có công thức

$$f(x, y) = 20 + x^2 + y^2 - 10 \cos(2\pi x) - 10 \cos(2\pi y), \quad x, y \in [-5.12, 5.12], \quad (2.2)$$

với cực tiểu toàn cục $f(0,0) = 0$. Hai số hạng cosin có chu kỳ bằng 1 nên tạo ra một mạng lưới cực tiểu cục bộ nằm xấp xỉ tại các điểm có tọa độ nguyên; trong miền $[-5.12, 5.12]^2$ có khoảng 121 cực tiểu cục bộ như vậy, bao quanh dày đặc cực tiểu toàn cục tại gốc tọa độ. Khó khăn ở đây là bài toán *đa cực trị* điển hình: một phương pháp cục bộ xuất phát từ điểm ngẫu nhiên gần như chắc chắn rơi vào giếng gần nhất rồi dừng lại, và giếng đó hầu như không bao giờ là giếng ở gốc tọa độ. Cái cứu GA ở đây là biên độ đột biến: với $\sigma = 0,08 \times 10,24 \approx 0,82$, một bước đột biến đủ lớn để nhảy qua vài ô lưới cực tiểu cục bộ (khoảng cách giữa hai giếng liền kề là 1), nên cá thể không bị giam trong một giếng duy nhất.



(a) Hàm Rastrigin trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.2: Hàm Rastrigin: mạng lưới cực tiểu cục bộ dày đặc quanh cực tiểu toàn cục, và diễn biến giá trị tốt nhất qua các thế hệ.

Hàm Rosenbrock có công thức

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad x \in [-5, 10], \quad y \in [-5, 10], \quad (2.3)$$

với cực tiểu toàn cục $f(1, 1) = 0$. Khác hẳn bốn hàm còn lại, hàm này *không* đa cực trị: trên miền hai biến nó chỉ có duy nhất một cực tiểu, nên không tồn tại bất kỳ bộ nào để thuật toán mắc vào. Khó khăn nằm ở hình dạng của đáy hàm. Số hạng $100(y - x^2)^2$ phạt rất nặng mọi sai lệch khỏi đường cong $y = x^2$, tạo thành một thung lũng hẹp hình quả chuối với vách hai bên dựng đứng, trong khi dọc theo lòng thung lũng giá trị hàm lại thay đổi rất chậm. Muốn tiến về $(1, 1)$, thuật toán phải thay đổi x và y một cách *phối hợp* sao cho y luôn bám theo x^2 ; đi lệch nhịp là lập tức đâm vào vách. Cụ thể, để đạt được $f < 10^{-3}$ thì đồng thời phải có $|1 - x| < 0,032$ và $|y - x^2| < 0,0032$ — một dải rất mỏng và cong. Cả hai toán tử của GA đều không phù hợp với yêu cầu này: lai ghép trộn nội suy theo *đường thẳng* nối hai cha mẹ, còn đột biến Gauss xê dịch từng tọa độ một cách *độc lập*, nên không toán tử nào bám được theo một đường cong. Nói cách khác, với Rosenbrock cái khó không phải là *tìm ra* vùng chứa nghiệm mà là *dò theo* đáy thung lũng sau khi đã vào được trong đó.

Hàm Ackley có công thức

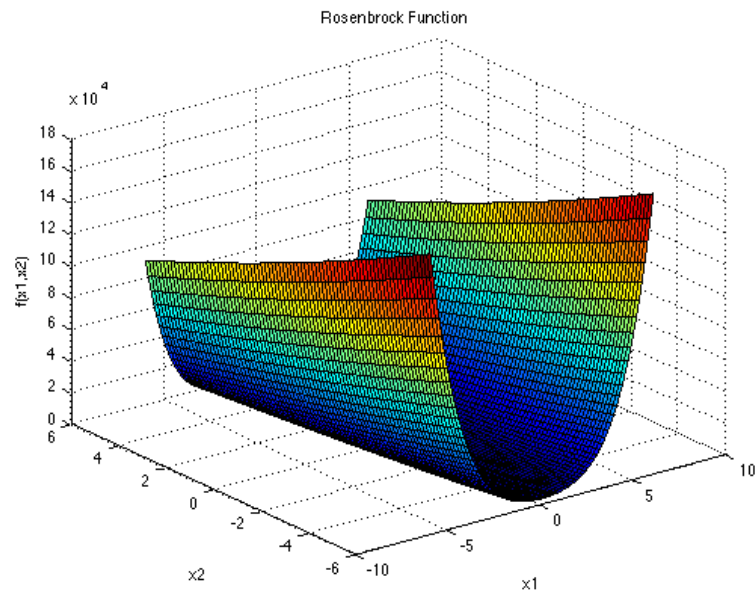
$$f(x, y) = -20 \exp\left(-0.2\sqrt{0.5(x^2 + y^2)}\right) - \exp(0.5 \cos(2\pi x) + 0.5 \cos(2\pi y)) + 20 + e, \quad (2.4)$$

với $x, y \in [-32.768, 32.768]$ và cực tiểu toàn cục $f(0, 0) = 0$. Hàm này kết hợp hai cấu trúc chồng lên nhau ở hai thang độ khác nhau: số hạng mũ thứ nhất tạo một cái phễu lớn dốc dần về gốc tọa độ, còn số hạng cosin phủ lên bề mặt phễu đó vô số gợn sóng cực tiểu cục bộ nhỏ. Cấu trúc phễu ở thang lớn là thông tin có ích — nó chỉ đúng hướng về nghiệm; các gợn sóng ở thang nhỏ mới là bẫy. Do đó Ackley kiểm tra khả năng *đọc được xu thế toàn cục mà không bị nhiễu cục bộ đánh lạc hướng*. Với GA, biên độ đột biến $\sigma = 0,08 \times 65,536 \approx 5,24$ lớn hơn nhiều so với bước sóng của gợn nhiễu (bằng 1), nên các gợn sóng gần như trong suốt đối với quá trình tìm kiếm.

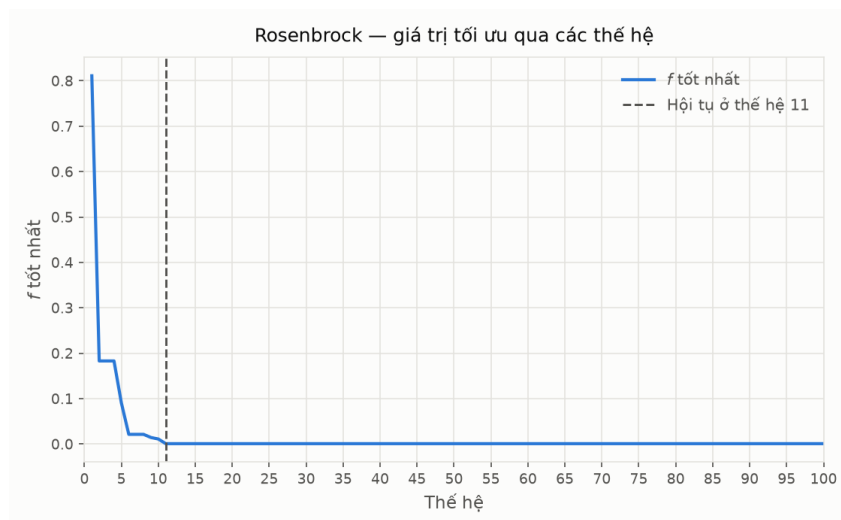
Hàm Schwefel có công thức

$$f(x, y) = 837.9658 - x \sin \sqrt{|x|} - y \sin \sqrt{|y|}, \quad x, y \in [-500, 500], \quad (2.5)$$

với cực tiểu toàn cục $f(420.9687, 420.9687) \approx 0$. Đây là hàm được thiết kế để *đánh lừa*: các cực tiểu cục bộ của nó phân bố không đều, và điều nguy hiểm là cực tiểu cục bộ tốt thứ nhì lại nằm rất xa cực tiểu toàn cục trong không gian tìm kiếm. Hệ quả là thông tin cục bộ dẫn đường sai — một thuật toán càng khai thác sớm và càng mạnh vùng đang có vẻ tốt thì càng bị kéo ra xa nghiệm đúng. Thêm vào đó, cực tiểu toàn cục nằm ở 420,97, tức sát biên 500 của miền tìm kiếm, nên những cài đặt cắt hoặc co

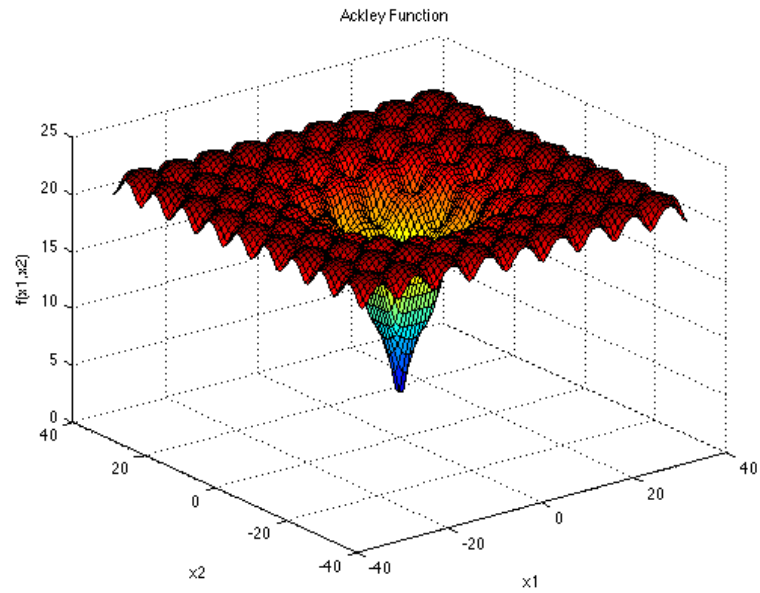


(a) Hàm Rosenbrock trong không gian ba chiều

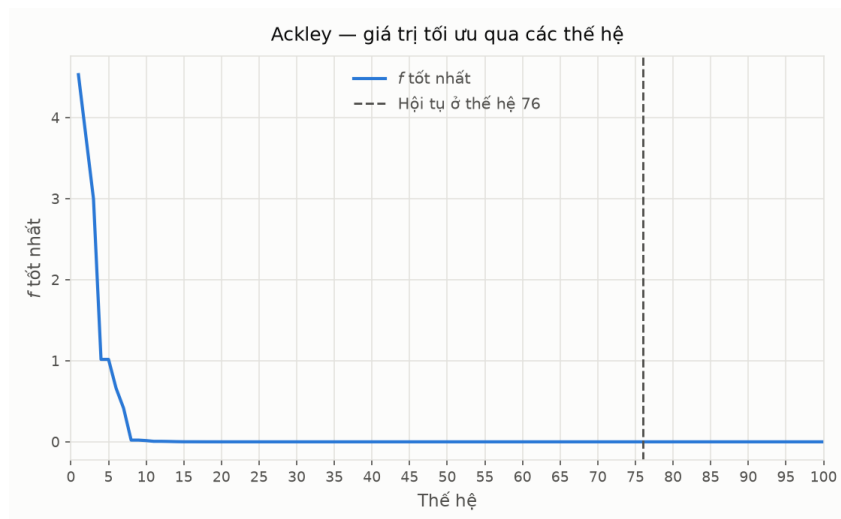


(b) Đường hội tụ của GA

Hình 2.3: Hàm Rosenbrock: thung lũng cong hẹp hình quả chuối, và diễn biến giá trị tốt nhất qua các thế hệ — GA chốt kỷ lục ngay ở thế hệ 11 rồi không cải thiện thêm trong 89 thế hệ còn lại.



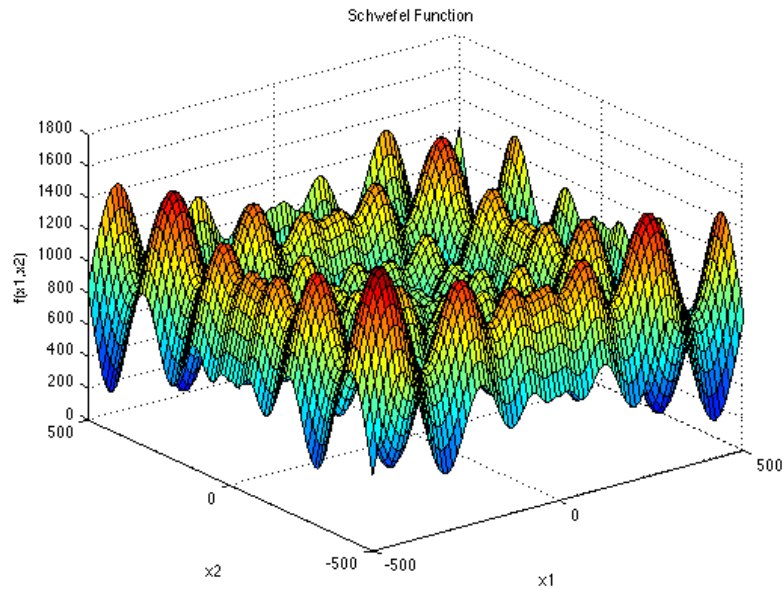
(a) Hàm Ackley trong không gian ba chiều



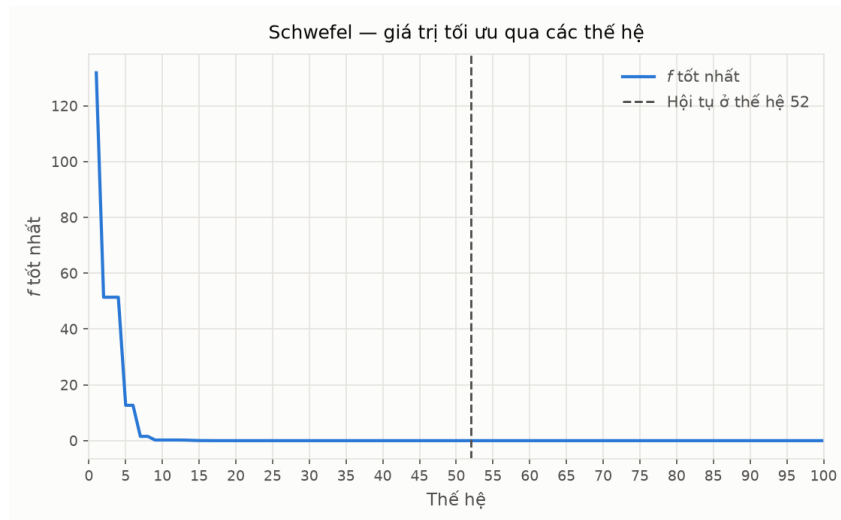
(b) Đường hội tụ của GA

Hình 2.4: Hàm Ackley: phễu lớn ở thang toàn cục phủ gọn sóng cực tiểu cục bộ ở thang nhỏ, và diễn biến giá trị tốt nhất qua các thế hệ.

cá thể về phía tâm miền sẽ bỏ sót nó. Hàm này vì vậy kiểm tra khả năng duy trì thăm dò trên diện rộng và không cam kết quá sớm vào một vùng.



(a) Hàm Schwefel trong không gian ba chiều



(b) Đường hội tụ của GA

Hình 2.5: Hàm Schwefel: cực tiểu toàn cục nằm gần biên miền tìm kiếm, xa vùng trông có vẻ tốt quanh gốc tọa độ, và diễn biến giá trị tốt nhất qua các thế hệ.

Với mỗi hàm, GA mã hóa số thực (lai ghép trộn BLX- α với $\alpha \in [-0,25; 1,25]$, đột biến Gauss với xác suất 0,15 trên mỗi gene và σ bằng 0,08 lần bề rộng miền, chọn lọc giải đấu theo thứ hạng với $k = 3$, giữ lại 2 cá thể tinh hoa mỗi thế hệ — theo Mục 1.3) được chạy một lần với quần thể 500 cá thể trong 100 thế hệ, hạt giống ngẫu nhiên cố định bằng 42 để kết quả tái lập được. Cá thể được cắt về miền tìm kiếm sau mỗi phép lai ghép và đột biến, vì ở đây miền đóng vai trò miền xác định của bài toán. Thuật toán không có tiêu chí dừng sớm: vòng lặp luôn chạy đủ 100 thế hệ. Cột *số thế hệ thỏa*

mãn trong Bảng 2.1 vì vậy không phải thể hệ mà thuật toán dừng lại, mà là thể hệ sớm nhất đạt được giá trị f^* cuối cùng — tức lần cuối cùng kỷ lục được cải thiện.

Bảng 2.1: Kết quả GA trên năm hàm benchmark không ràng buộc

Hàm	Thời gian (s)	Số thế hệ thỏa mãn	f^*	x^*	y^*
Easom	1,3619	51	$-1,0000000000$	3,141593	3,141593
Rastrigin	1,3876	45	$0,0000000000$	$8,00 \times 10^{-10}$	$-2,11 \times 10^{-9}$
Rosenbrock	1,4194	11	0,0009051359	1,008839	1,014881
Ackley	1,5411	76	$0,0000000000$	$-9,22 \times 10^{-17}$	$-2,99 \times 10^{-16}$
Schwefel	1,4160	52	$2,5455 \times 10^{-5}$	420,968746	420,968746

GA tìm đúng, hoặc gần đúng đến sai số không đáng kể, cực tiểu toàn cục trên cả năm hàm — kể cả Easom và Schwefel, hai hàm được thiết kế riêng để đánh lừa các thuật toán chỉ khai thác thông tin cục bộ quanh điểm hiện tại. Kết quả này minh chứng trực tiếp cho ưu điểm cốt lõi của tìm kiếm dựa trên quần thể đã trình bày ở Mục 1.1: vì GA duy trì 500 cá thể trải rộng trên không gian tìm kiếm thay vì một điểm duy nhất, nó không phụ thuộc vào việc điểm khởi tạo có nằm gần cực tiểu toàn cục hay không, khác với các phương pháp gradient cổ điển. Với Rastrigin và Ackley, GA về đúng 0 trong giới hạn biểu diễn số thực dấu phẩy động.

Rosenbrock là ngoại lệ duy nhất, và cách nó khác biệt lại soi sáng đúng điểm mạnh cũng như điểm yếu của GA. Sai số $9,05 \times 10^{-4}$ của nó lớn hơn bốn hàm còn lại nhiều bậc độ lớn, mặc dù đây là hàm *duy nhất* trong năm hàm không hề có bất kỳ cực tiểu cục bộ. Đường hội tụ ở Hình 2.3 cho thấy rõ cơ chế: giá trị tốt nhất tụt rất nhanh trong mười thế hệ đầu rồi chốt lại ở thế hệ 11 và đứng yên suốt 89 thế hệ còn lại. Nghĩa là GA lọt vào đúng thung lũng gần như tức thì, nhưng sau đó không nhích thêm được nữa. Đối chiếu với Ackley — hàm mà GA cải thiện kỷ lục rải rác cho tới tận thế hệ 76 — có thể thấy hai kiểu khó khăn hoàn toàn khác nhau: Rastrigin, Ackley, Easom và Schwefel khó ở chỗ *tìm ra* đúng vùng chứa nghiệm, còn Rosenbrock khó ở chỗ *dò theo* một cấu trúc cong sau khi đã tìm ra vùng đó.

Sự phân đôi này phản ánh bản chất của các toán tử tiến hóa ngẫu nhiên đẳng hướng: chúng rất mạnh trong việc định vị một vùng rộng trên không gian tìm kiếm, nhưng yếu trong việc tinh chỉnh dọc theo một hướng ưu tiên hẹp. Đây chính là lý do các thuật toán lai ghép GA với tìm kiếm cục bộ (memetic algorithm) tồn tại — và cũng là cách tiếp cận được áp dụng cho bài toán người du lịch ở Mục 2.3, nơi GA thuần túy được bổ sung thêm bước tối ưu cục bộ 2-opt để xử lý đúng phần việc mà lai ghép và đột biến ngẫu nhiên làm không tốt.

2.2 Tối ưu có ràng buộc: Bộ chuẩn CEC 2006

Ở lớp bài toán không ràng buộc trình bày tại Mục 2.1, mọi điểm thuộc miền tìm kiếm đều là một nghiệm hợp lệ, nên toàn bộ công sức của thuật toán dành cho việc cải thiện hàm mục tiêu. Khi có thêm ràng buộc, tình thế thay đổi về bản chất: phần lớn không gian tìm kiếm trở thành vùng không sử dụng được, và thuật toán phải xác định được miền khả thi trước khi có thể bàn tới chuyện tối ưu. Một giá trị hàm mục tiêu nhỏ đạt được tại điểm vi phạm ràng buộc không mang ý nghĩa gì, thậm chí còn gây hiểu nhầm vì nó thường nhỏ hơn giá trị tối ưu thực sự.

Để đánh giá GA trên lớp bài toán này, thực nghiệm sử dụng bộ chuẩn CEC 2006 của Liang và cộng sự — tập hợp hai mươi bốn bài toán tối ưu có ràng buộc được công bố kèm nghiệm tối ưu, hiện là thước đo tham chiếu phổ biến trong lĩnh vực tính toán tiến hóa. So với các bài toán tự đặt, một bộ chuẩn đã công bố mang lại hai lợi thế. Thứ nhất, nghiệm đúng đã được xác lập nên sai lệch của GA đo được trực tiếp thay vì phải dựa vào một phương pháp đối chứng. Thứ hai, kết quả thu được có thể đặt cạnh các công trình khác cùng sử dụng bộ chuẩn này.

Năm bài toán được chọn là g01, g06, g08, g11 và g15. Tiêu chí lựa chọn là bao quát các dạng ràng buộc khác nhau, đồng thời trải rộng về mức độ chặt hẹp của miền khả thi và về vị trí tương đối giữa nghiệm tối ưu và biên của miền đó. Bảng 2.2 tóm tắt đặc điểm của năm bài toán, trong đó ρ là tỉ lệ ước lượng giữa thể tích miền khả thi và thể tích miền tìm kiếm, còn cột cuối cho biết số ràng buộc đạt đẳng thức tại nghiệm tối ưu.

Bảng 2.2: Đặc điểm năm bài toán được chọn từ bộ chuẩn CEC 2006

Bài toán	Số biến	Dạng hàm	ρ	Ràng buộc	Ràng buộc active
g01	13	bậc hai	0,0111%	9 bất đẳng thức tuyến tính	6
g06	2	bậc ba	0,0066%	2 bất đẳng thức phi tuyến	2
g08	2	phi tuyến	0,8560%	2 bất đẳng thức phi tuyến	0
g11	2	bậc hai	0,0000%	1 đẳng thức phi tuyến	1
g15	3	bậc hai	0,0000%	2 đẳng thức, 1 tuyến tính	2

Bộ chuẩn quy định rõ thế nào là một nghiệm khả thi. Với ràng buộc bất đẳng thức, điều kiện là $g_i(x) \leq 0$ được thỏa mãn tuyệt đối. Với ràng buộc đẳng thức, điều kiện được nối thành $|h_j(x)| - \varepsilon \leq 0$ với $\varepsilon = 10^{-4}$. Sự phân biệt này là cần thiết chứ không tùy tiện: tập nghiệm của một phương trình $h_j(x) = 0$ có thể tích bằng không trong không gian tìm kiếm, nên không một thuật toán số nào chạm đúng vào nó được. Bản thân hai nghiệm tối ưu công bố của g11 và g15 cũng vi phạm ràng buộc đẳng thức đúng bằng 10^{-4} , tức nằm sát ranh giới mà dung sai cho phép. Ngược lại, ràng buộc bất đẳng thức được thỏa mãn trên cả một vùng có thể tích dương nên việc đòi hỏi vi phạm bằng không là hợp lý. Toàn bộ kết quả báo cáo dưới đây đều có mức vi phạm bằng 0 theo đúng định nghĩa này.

Về phương pháp, GA mã hóa số thực xử lý ràng buộc bằng quy tắc khả thi của Deb kết hợp ngưỡng ε giảm dần đã trình bày ở Mục 1.3, với quần thể 1000 cá thể chạy trong 500 thế hệ. Tiếp sau GA là một bước tinh chỉnh cục bộ theo nguyên lý tìm kiếm trực tiếp: xuất phát từ nghiệm tốt nhất mà GA tìm được, thuật toán dò lần lượt theo từng trục tọa độ với một bước dịch chuyển ban đầu, chỉ chấp nhận những điểm vừa còn khả thi vừa có giá trị hàm mục tiêu nhỏ hơn; khi không còn hướng nào cải thiện được nữa thì bước dịch chuyển được giảm đi một nửa và quá trình lặp lại. Cách kết hợp một thuật toán tiến hóa với một thủ tục tìm kiếm cục bộ như vậy chính là mô hình thuật toán ghi nhớ (memetic algorithm), cũng là cách tiếp cận được dùng cho bài toán người du lịch ở Mục 2.3. Vai trò phân công giữa hai giai đoạn rất rõ: GA đảm nhiệm việc định vị vùng chứa nghiệm trên toàn miền tìm kiếm, còn bước tinh chỉnh đảm nhiệm việc tiếp cận chính xác những nghiệm nằm sát biên của miền khả thi. Kết quả của cả hai giai đoạn đều được báo cáo để thấy rõ phần đóng góp của từng bước.

2.2.1 Bài toán g01

Bài toán g01 có hàm mục tiêu bậc hai trên mười ba biến, chịu chín ràng buộc bất đẳng thức đều tuyến tính:

$$\begin{aligned} \min_{x \in \mathbb{R}^{13}} \quad & f(x) = 5 \sum_{i=1}^4 x_i - 5 \sum_{i=1}^4 x_i^2 - \sum_{i=5}^{13} x_i \\ \text{với điều kiện} \quad & 2x_1 + 2x_2 + x_{10} + x_{11} - 10 \leq 0 \\ & 2x_1 + 2x_3 + x_{10} + x_{12} - 10 \leq 0 \\ & 2x_2 + 2x_3 + x_{11} + x_{12} - 10 \leq 0 \\ & -8x_1 + x_{10} \leq 0 \\ & -8x_2 + x_{11} \leq 0 \\ & -8x_3 + x_{12} \leq 0 \\ & -2x_4 - x_5 + x_{10} \leq 0 \\ & -2x_6 - x_7 + x_{11} \leq 0 \\ & -2x_8 - x_9 + x_{12} \leq 0 \end{aligned} \tag{2.6}$$

với miền biến thiên $0 \leq x_i \leq 1$ cho $i = 1, \dots, 9$, $0 \leq x_i \leq 100$ cho $i = 10, 11, 12$ và $0 \leq x_{13} \leq 1$. Nghiệm tối ưu công bố là

$$x^* = (1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1), \quad f(x^*) = -15,$$

tại đó sáu trong chín ràng buộc đạt đẳng thức.

Đây là bài toán có số biến lớn nhất trong năm bài. Điểm thuận lợi nằm ở chỗ mọi ràng buộc đều tuyến tính, nên miền khả thi là một đa diện lồi và vùng lân cận nghiệm

tối ưu vẫn còn thể tích đáng kể. Nhờ vậy quần thể di chuyển tương đối dễ dàng ngay cả khi đã tiến sát nghiệm.

Bảng 2.3: Kết quả trên bài toán g01

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm
-15,000000	-14,999204	-15,000000	$1,77 \times 10^{-12}$	0

GA đơn thuần đạt sai lệch tương đối $5,3 \times 10^{-5}$, một kết quả tốt đối với không gian mười ba chiều. Bước tinh chỉnh đưa sai lệch xuống $1,77 \times 10^{-12}$, tức trùng với nghiệm công bố trong giới hạn biểu diễn số thực dấu phẩy động.

2.2.2 Bài toán g06

Bài toán g06 chỉ có hai biến nhưng hàm mục tiêu bậc ba và cả hai ràng buộc đều phi tuyến:

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = (x_1 - 10)^3 + (x_2 - 20)^3 \\ \text{với điều kiện} \quad & -(x_1 - 5)^2 - (x_2 - 5)^2 + 100 \leq 0 \\ & (x_1 - 6)^2 + (x_2 - 5)^2 - 82,81 \leq 0 \end{aligned} \quad (2.7)$$

với $13 \leq x_1 \leq 100$ và $0 \leq x_2 \leq 100$. Nghiệm tối ưu công bố là

$$x^* = (14,095000; 0,842961), \quad f(x^*) = -6961,813876,$$

tại đó cả hai ràng buộc đều đạt đẳng thức.

Hai ràng buộc mô tả phần bên ngoài một đường tròn và phần bên trong một đường tròn khác, nên miền khả thi là một dải hình lưỡi liềm rất hẹp, chỉ chiếm 0,0066% thể tích miền tìm kiếm. Nghiệm tối ưu nằm đúng tại giao điểm của hai đường tròn, tức một điểm không chiều. Bên cạnh đó, chuẩn của gradient hàm mục tiêu tại nghiệm xấp xỉ 1102, lớn hơn hai bậc độ lớn so với các bài toán còn lại. Hệ quả là một sai lệch nhỏ về vị trí bị khuếch đại thành sai lệch lớn về giá trị hàm mục tiêu. Đây là bài toán duy nhất trong năm bài hội tụ đồng thời cả hai yếu tố bất lợi: miền khả thi cực hẹp quanh nghiệm và hàm mục tiêu rất dốc.

Bảng 2.4: Kết quả trên bài toán g06

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm
-6961,813876	-6898,462524	-6961,813876	$2,64 \times 10^{-8}$	0

GA đơn thuần dừng lại ở -6898,462524, cách nghiệm công bố 63,35 đơn vị, tương ứng sai lệch tương đối $9,1 \times 10^{-3}$. Cần lưu ý rằng con số này phản ánh độ dốc của hàm mục tiêu nhiều hơn là chất lượng định vị của GA: quy đổi qua chuẩn gradient,

sai lệch đó tương ứng với sai số vị trí chỉ khoảng 0,06% bề rộng miền tìm kiếm. Bước tinh chỉnh cục bộ đưa kết quả về $2,64 \times 10^{-8}$, cải thiện hơn chín bậc độ lớn. Đây là bài toán mà việc kết hợp tìm kiếm cục bộ phát huy tác dụng rõ rệt nhất.

2.2.3 Bài toán g08

Bài toán g08 có hàm mục tiêu chứa hàm lượng giác và phân thức:

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = -\frac{\sin^3(2\pi x_1) \sin(2\pi x_2)}{x_1^3(x_1 + x_2)} \\ \text{với điều kiện} \quad & x_1^2 - x_2 + 1 \leq 0 \\ & 1 - x_1 + (x_2 - 4)^2 \leq 0 \end{aligned} \quad (2.8)$$

với $0 \leq x_1 \leq 10$ và $0 \leq x_2 \leq 10$. Nghiệm tối ưu công bố là

$$x^* = (1,227971; 4,245373), \quad f(x^*) = -0,095825.$$

Điểm khác biệt căn bản của bài toán này là nghiệm tối ưu nằm hẳn bên trong miền khả thi, không ràng buộc nào đạt đẳng thức. Do đó nghiệm là một điểm dừng của hàm mục tiêu và gradient tại đó bằng không, khiến sai lệch về vị trí hầu như không chuyển thành sai lệch về giá trị hàm. Miền khả thi cũng rộng rãi hơn hẳn bốn bài còn lại, chiếm 0,8560% miền tìm kiếm.

Bảng 2.5: Kết quả trên bài toán g08

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm
-0,095825	-0,095825	-0,095825	$2,78 \times 10^{-17}$	0

GA đạt nghiệm công bố ngay ở giai đoạn đầu với sai lệch $2,78 \times 10^{-17}$, tức đã chạm ngưỡng chính xác của số thực dấu phẩy động, và bước tinh chỉnh không còn gì để cải thiện. Kết quả này minh họa rằng độ khó của một bài toán có ràng buộc không nằm ở tính phi tuyến của hàm mục tiêu mà nằm ở vị trí của nghiệm so với biên miền khả thi.

2.2.4 Bài toán g11

Bài toán g11 có dạng đơn giản nhất về hình thức nhưng chứa một ràng buộc đẳng thức phi tuyến:

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = x_1^2 + (x_2 - 1)^2 \\ \text{với điều kiện} \quad & x_2 - x_1^2 = 0 \end{aligned} \quad (2.9)$$

với $-1 \leq x_1 \leq 1$ và $-1 \leq x_2 \leq 1$. Nghiệm tối ưu công bố là

$$x^* = (-0,707036; 0,500000), \quad f(x^*) = 0,7499.$$

Miền khả thi ở đây là một đường cong parabol trong mặt phẳng, tức một tập có diện tích bằng không. Không một điểm sinh ngẫu nhiên nào rơi trúng miền này, nên toàn bộ việc tiếp cận miền khả thi phụ thuộc vào cơ chế nói lỏng ngưỡng ε của thuật toán.

Bài toán này cho phép kiểm chứng nghiệm bằng giải tích. Thế $x_2 = x_1^2$ vào hàm mục tiêu và đặt $u = x_1^2 \geq 0$, ta được $u + (u - 1)^2 = u^2 - u + 1$, đạt cực tiểu tại $u = 1/2$ với giá trị đúng bằng 0,75. Như vậy cực tiểu chính xác của bài toán là 0,75, trong khi giá trị công bố 0,7499 lại nhỏ hơn con số đó. Nguyên nhân là nghiệm công bố không nằm trên đường cong mà lệch khỏi nó đúng bằng dung sai $\varepsilon = 10^{-4}$, và độ lệch ấy được tận dụng theo hướng làm giảm hàm mục tiêu.

Bảng 2.6: Kết quả trên bài toán g11

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm
0,749900	0,750268	0,750259	$3,59 \times 10^{-4}$	0

GA hội tụ về 0,750259, tức cách cực tiểu giải tích 0,75 một khoảng $2,6 \times 10^{-4}$ và cách giá trị công bố $3,59 \times 10^{-4}$. Phần chênh lệch còn lại mang tính cấu trúc chứ không phải do thuật toán hội tụ kém: muốn đạt được đúng con số 0,7499 thì nghiệm phải nằm ở rìa ngoài của dải dung sai, điều mà một thuật toán tìm nghiệm khả thi không hướng tới. Bước tinh chỉnh cải thiện không đáng kể vì miền khả thi là một dải cong rất hẹp, trong đó các bước dịch chuyển song song với trục tọa độ nhanh chóng đi ra ngoài miền và bị loại.

2.2.5 Bài toán g15

Bài toán g15 có ba biến cùng hai ràng buộc đẳng thức, một phi tuyến và một tuyến tính:

$$\begin{aligned} \min_{x \in \mathbb{R}^3} \quad & f(x) = 1000 - x_1^2 - 2x_2^2 - x_3^2 - x_1x_2 - x_1x_3 \\ \text{với điều kiện} \quad & x_1^2 + x_2^2 + x_3^2 - 25 = 0 \\ & 8x_1 + 14x_2 + 7x_3 - 56 = 0 \end{aligned} \tag{2.10}$$

với $0 \leq x_i \leq 10$ cho $i = 1, 2, 3$. Nghiệm tối ưu công bố là

$$x^* = (3,512128; 0,216988; 3,552179), \quad f(x^*) = 961,715022.$$

Hai ràng buộc đẳng thức mô tả một mặt cầu và một mặt phẳng, giao của chúng

là một đường tròn trong không gian ba chiều. Xét về thể tích, đây là miền khả thi chặt hẹp nhất trong năm bài toán. Bù lại, hàm mục tiêu khá thoải: chuẩn gradient tại nghiệm chỉ khoảng 15,8, nên sai lệch về vị trí không bị khuếch đại mạnh như ở g06.

Bảng 2.7: Kết quả trên bài toán g15

f^* công bố	f^* (GA)	f^* (GA + tinh chỉnh)	Sai lệch	Vi phạm
961,715022	961,715374	961,715308	$2,86 \times 10^{-4}$	0

Sai lệch tương đối chỉ $3,0 \times 10^{-7}$, thuộc nhóm kết quả tốt nhất trong năm bài toán mặc dù miền khả thi hẹp nhất. Điều đáng chú ý là kết quả này đạt được ngay từ giai đoạn GA; cũng như ở g11, bước tinh chỉnh cục bộ không phát huy nhiều tác dụng khi miền khả thi là một đường cong hẹp.

2.2.6 Nhận xét chung

Bảng 2.8 tổng hợp kết quả trên cả năm bài toán. Mọi nghiệm báo cáo đều khả thi tuyệt đối theo định nghĩa của bộ chuẩn, với mức vi phạm bằng 0.

Bảng 2.8: Tổng hợp kết quả GA trên năm bài toán chuẩn CEC 2006

Bài toán	f^* công bố	f^* (GA)	f^* (GA) + tinh chỉnh	Sai lệch tương đối	Thời gian (s)
g01	-15,000000	-14,999204	-15,000000	$1,2 \times 10^{-13}$	38,8
g06	-6961,813876	-6898,462524	-6961,813876	$3,8 \times 10^{-12}$	19,9
g08	-0,095825	-0,095825	-0,095825	$2,9 \times 10^{-16}$	20,0
g11	0,749900	0,750268	0,750259	$4,8 \times 10^{-4}$	17,3
g15	961,715022	961,715374	961,715308	$3,0 \times 10^{-7}$	29,6

Kết quả cho thấy độ khó của một bài toán tối ưu có ràng buộc đối với GA không được quyết định bởi số biến hay bậc của hàm mục tiêu, mà bởi hai yếu tố thuộc về hình học của bài toán tại lân cận nghiệm tối ưu.

Yếu tố thứ nhất là thể tích miền khả thi quanh nghiệm. Khi nghiệm nằm sâu bên trong miền khả thi như ở g08, quần thể di chuyển tự do và GA đạt độ chính xác giới hạn của số thực ngay từ giai đoạn tiến hóa. Khi nghiệm nằm tại một đỉnh của đa diện lồi như ở g01, vùng lân cận vẫn còn thể tích đáng kể nên GA tiếp cận tốt. Ngược lại, khi nghiệm nằm tại giao điểm của hai mặt cong như ở g06, hoặc trên một đường cong có thể tích bằng không như ở g11 và g15, các phép lai ghép và đột biến ngẫu nhiên hầu như luôn sinh ra cá thể bất khả thi và bị loại bỏ, khiến quá trình cải thiện chững lại.

Yếu tố thứ hai là độ dốc của hàm mục tiêu tại nghiệm, đại lượng quyết định một sai lệch về vị trí được khuếch đại bao nhiêu lần thành sai lệch về giá trị hàm. Chuẩn gradient tại nghiệm của g06 lớn gấp khoảng bảy mươi lần so với g15 và gấp một trăm

lần so với g01, giải thích vì sao cùng một mức chính xác về vị trí lại cho ra sai lệch chênh nhau nhiều bậc độ lớn giữa các bài toán.

Hai yếu tố này độc lập với nhau, và g06 là bài toán duy nhất mà cả hai cùng bất lợi. Đó cũng là bài toán mà giai đoạn tinh chỉnh cục bộ mang lại cải thiện lớn nhất, đưa sai lệch từ 63,35 xuống $2,64 \times 10^{-8}$. Nhìn rộng hơn, sự phân công giữa hai giai đoạn đã được xác nhận qua thực nghiệm: GA giải quyết phần việc định vị vùng chứa nghiệm trên một không gian mà phần lớn là bất khả thi, còn tìm kiếm cục bộ giải quyết phần việc tiếp cận chính xác nghiệm nằm sát biên. Không giai đoạn nào thay thế được giai đoạn kia, và chi phí tính toán của giai đoạn tinh chỉnh không đáng kể so với giai đoạn tiến hóa.

Riêng với g11 và g15, phần sai lệch còn lại chủ yếu bắt nguồn từ dung sai $\varepsilon = 10^{-4}$ dành cho ràng buộc đẳng thức chứ không phải từ năng lực hội tụ của thuật toán. Như đã phân tích ở Mục 2.2.4, giá trị công bố của g11 còn nhỏ hơn cực tiểu chính xác của bài toán, nên khoảng cách giữa kết quả thu được và giá trị công bố không phản ánh sai số của phương pháp.

2.3 Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ

Bài toán người du lịch (Traveling Salesman Problem — TSP) được phát biểu như sau: cho n địa điểm và ma trận khoảng cách $D = (d_{ij})_{n \times n}$ giữa mọi cặp địa điểm, tìm một hoán vị $\pi = (\pi_1, \dots, \pi_n)$ của n địa điểm sao cho tổng độ dài lộ trình khép kín đi qua tất cả các địa điểm đúng một lần là nhỏ nhất:

$$\min_{\pi} L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1}. \quad (2.11)$$

Bài toán này được mã hóa dưới dạng hoán vị đã giới thiệu ở Mục 1.2: mỗi cá thể của GA chính là một hoán vị π , dùng lai ghép Order Crossover (OX) và đột biến Swap (Mục 1.3) thay cho các toán tử số thực ở hai mục trước.

Để đánh giá khách quan trên dữ liệu thực thay vì các ví dụ tự tạo quy mô nhỏ, ba bộ dữ liệu chuẩn từ thư viện benchmark TSPLIB95 [9] được sử dụng: berlin52 ($n = 52$ địa điểm tại Berlin), rd100 ($n = 100$ địa điểm ngẫu nhiên), và ch150 ($n = 150$ địa điểm). Mỗi bộ dữ liệu đều có kèm theo lộ trình tối ưu đã được công bố, dùng làm mốc đối chiếu khách quan thay cho phương pháp vét cạn — vốn cần thử $(n - 1)!$ hoán vị và trở nên bất khả thi về mặt tính toán ngay từ n vài chục.

Thử nghiệm ban đầu với GA hoán vị thuần túy (chỉ dùng OX và Swap) cho kết quả chênh lệch rất lớn so với nghiệm tối ưu đã công bố, từ 20% đến 84% tùy bộ dữ liệu và tăng dần theo số địa điểm n . Nguyên nhân là hai toán tử OX và Swap không có cơ

chế sửa các đoạn đường bị cắt chéo nhau trong lộ trình — một lộ trình càng có nhiều địa điểm thì càng dễ chứa những đoạn cắt chéo như vậy, và OX/Swap chỉ có thể tạo ra một cấu hình mới ngẫu nhiên chứ không chủ động “gỡ” đoạn cắt chéo đó. Để khắc phục hạn chế này, một chiến lược lai giữa GA và tìm kiếm cục bộ (*memetic algorithm*) được áp dụng: sau mỗi thế hệ, một tỉ lệ cá thể tốt nhất trong quần thể được tinh chỉnh thêm bằng phép tìm kiếm cục bộ **2-opt** — xét lần lượt từng cặp cạnh trong lộ trình, hoán đổi hai cạnh đó nếu việc hoán đổi làm giảm tổng độ dài — trước khi tiếp tục vòng lặp tiến hóa (chọn lọc, lai ghép, đột biến) như bình thường. Với tỉ lệ tinh chỉnh bằng khoảng 5% quần thể mỗi thế hệ, cả ba bộ dữ liệu đều đạt đúng lộ trình tối ưu đã công bố, như trình bày ở Bảng 2.9.

Bảng 2.9: Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95

Bộ dữ liệu	Số địa điểm (n)	Nghiem tối ưu	GA	Đạt được ở thế hệ
berlin52	52	7542	7542	8/50
rd100	100	7910	7910	48/50
ch150	150	6528	6528	37/50

Trong quá trình hiệu chỉnh tham số, một quan sát quan trọng được ghi nhận: tỉ lệ cá thể được tinh chỉnh bằng 2-opt cần được đặt theo *phần trăm* của quần thể chứ không phải theo một số lượng cố định. Cụ thể, khi giữ nguyên số lượng cá thể được tinh chỉnh ở một con số tuyệt đối nhưng tăng kích thước quần thể lên, kết quả trở nên tệ hơn so với quần thể nhỏ — nguyên nhân là vì trong một quần thể lớn, số cá thể đã qua tinh chỉnh chiếm tỉ lệ quá nhỏ nên khó lan gen tốt ra phần còn lại của quần thể thông qua phép chọn lọc. Quan sát này cho thấy các tham số của một chiến lược lai giữa GA và tìm kiếm cục bộ không thể được chọn độc lập với nhau, mà phải cân chỉnh tương thích với quy mô quần thể.

2.4 Hồi quy Ký hiệu bằng Lập trình Di truyền

Ứng dụng cuối cùng minh họa Lập trình Di truyền (Mục 1.5) cho bài toán hồi quy ký hiệu trên dữ liệu vật lý thực: các phép đo mật độ hạt, vận tốc, và từ trường của gió mặt trời do vệ tinh WIND của NASA ghi nhận. Ba thành phần từ trường đo được là B_X, B_Y, B_Z , và mật độ hạt đo được là n . Từ bốn đại lượng này, hai đại lượng vật lý phái sinh được chọn làm mục tiêu dự đoán cho GP:

$$|B|^2 = B_X^2 + B_Y^2 + B_Z^2 \quad (\text{bình phương độ lớn từ trường}), \quad (2.12)$$

$$V_A^2 \propto \frac{|B|^2}{n} \quad (\text{vận tốc Alfvén bình phương}). \quad (2.13)$$

Đại lượng thứ nhất là một quan hệ tổng bình phương (dạng Pythagore) giữa ba biến đo được; đại lượng thứ hai kết hợp thêm một phép chia cho biến mật độ hạt.

Điểm mấu chốt khiến hai đại lượng này trở thành phép thử công bằng cho GP là: quan hệ tổng bình phương ở Phương trình (2.12) *không thể* tuyến tính hóa bằng phép biến đổi logarit như các quan hệ dạng lũy thừa thông thường, vì B_X, B_Y, B_Z có thể mang giá trị âm nên không lấy logarit được. Do đó, Hồi quy Tuyến tính áp dụng trực tiếp trên ba đặc trưng thô B_X, B_Y, B_Z gần như chắc chắn không thể biểu diễn được cấu trúc bậc hai này, vì mô hình tuyến tính chỉ có thể biểu diễn một tổ hợp tuyến tính của chính các đặc trưng đó, không tự sinh ra được số hạng bậc hai. Ngược lại, GP có thể tự xây dựng số hạng bậc hai bằng cách nhân một biến với chính nó ngay trong cây biểu thức, mà không cần được cung cấp trước các đặc trưng B_X^2, B_Y^2, B_Z^2 như một phép biến đổi thủ công. Kết quả đối chiếu hai phương pháp trên tập kiểm tra được trình bày ở Bảng 2.10.

Bảng 2.10: Kết quả Hồi quy Tuyến tính và GP trên hai đại lượng vật lý phái sinh

Đại lượng	Phương pháp	MAE	RMSE	R^2
$ B ^2$	Hồi quy Tuyến tính	15.1284	19.9825	0.4784
	GP (hồi quy ký hiệu)	4.1820	6.0779	0.9517
V_A^2	Hồi quy Tuyến tính	2.7376	3.6804	0.4629
	GP (hồi quy ký hiệu)	1.0554	1.5486	0.9049

Kết quả cho thấy chênh lệch rõ rệt và nhất quán trên cả hai đại lượng. Hồi quy Tuyến tính chỉ giải thích được khoảng 46% đến 48% phương sai của dữ liệu ($R^2 \approx 0.46\text{--}0.48$), trong khi GP đạt trên 90% ($R^2 \approx 0.90\text{--}0.95$); đồng thời sai số dự đoán (MAE, RMSE) của GP chỉ còn chưa đến một phần ba so với Hồi quy Tuyến tính ở cả hai đại lượng. Đây là kết quả trái ngược hẳn với các bài toán hồi quy dạng lũy thừa từng được thử nghiệm trong quá trình thực hiện tiểu luận — ví dụ định luật Kepler III hay áp suất động của gió mặt trời — là những quan hệ trở thành tuyến tính tuyệt đối sau khi biến đổi logarit, khiến Hồi quy Tuyến tính ở các bài toán đó luôn đạt hoặc vượt GP. Sự tương phản giữa hai nhóm kết quả khẳng định đúng giả thuyết ban đầu: lợi thế thực sự của GP so với hồi quy cổ điển chỉ bộc lộ rõ khi quan hệ cần khám phá có cấu trúc phi tuyến mà phép biến đổi đặc trưng thủ công thông thường, như logarit, không xử lý được.

2.5 Tổng kết Chương

Bốn thực nghiệm trên minh họa một đặc điểm xuyên suốt của Thuật toán Di truyền: tính linh hoạt cao nhờ khả năng thích ứng với nhiều cách mã hóa khác nhau — số thực ở Mục 2.1 và 2.2, hoán vị ở Mục 2.3, và cây biểu thức ở Mục 2.4. Đổi lại tính

linh hoạt đó là hiệu quả thấp hơn các phương pháp chuyên dụng khi bài toán có sẵn một cấu trúc mà phương pháp đó khai thác được: ở năm bài toán chuẩn CEC 2006 tại Mục 2.2, GA phải bổ sung một bước tìm kiếm cục bộ mới tiếp cận được những nghiệm nằm sát biên miền khả thi, phần việc mà một phương pháp dựa trên gradient xử lý trực tiếp nhờ khai thác tính khả vi của hàm mục tiêu và ràng buộc. Ngược lại, ở những bài toán mà cấu trúc đó không có sẵn, hoặc phương pháp cổ điển tương ứng không còn đủ mạnh — bài toán người du lịch quy mô lớn cần bổ sung tìm kiếm cục bộ mới đạt được nghiệm tối ưu, hay quan hệ phi tuyến dạng tổng bình phương mà Hồi quy Tuyến tính không thể biểu diễn được — GA và GP thể hiện rõ giá trị thực tiễn của một phương pháp tìm kiếm không đặt giả định trước về cấu trúc bài toán.

Tài liệu tham khảo

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT Press.
4. Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
5. Katoch, S., Chauhan, S. S., & Kumar, V. (2020). A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications*, 80, 8091–8126.
6. Whitley, D., Starkweather, T., & Bogart, C. (1990). Genetic algorithms and neural networks: optimizing connections and connectivity. *Parallel Computing*, 14, 347–361.
7. Takahama, T., & Sato, S. (2010). Constrained optimization by the ε constrained differential evolution with gradient-based mutation and feasible elites. *IEEE Congress on Evolutionary Computation (CEC)*.
8. Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.
9. Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376–384.